

Renault Care Assistant — Build Notes

Conversation summary: Pipecat voice-agent system prompt design, product-expert mode tool implementation, and LLMContext / system_instruction reference.

1. Overview

The goal: extend an existing Pipecat-based Renault Care voice assistant (Asterisk/AudioSocket transport, Google Vertex LLM, existing end_conversation and transfer_to_agent tools) with a new "Product Expert" mode — a tool-triggered system-prompt swap that lets the assistant answer detailed Duster/Kiger spec questions using brochure data, then swap back to the base routing prompt.

2. New Tool Schemas

Two FunctionSchema definitions, following the same pattern as the existing END_CONVERSATION_TOOL / TRANSFER_AGENT_TOOL:

```
from pipecat.adapters.schemas.function_schema import FunctionSchema

ENTER_PRODUCT_EXPERT_TOOL = FunctionSchema(
    name="enter_product_expert_mode",
    description=(
        "CALL THIS IMMEDIATELY when the user asks about vehicle specs, features, "
        "engine options, dimensions, safety, interior, or comparisons for the "
        "Renault Duster or Renault Kiger. "
        "Do NOT attempt to answer from memory first - call this function BEFORE "
        "generating any spoken product content, so you have brochure data loaded. "
        "Trigger phrases: 'engine', 'mileage', 'features', 'boot space', 'safety rating', "
        "'Duster', 'Kiger', 'variant', 'price', 'specifications'."
    ),
    properties={
        "model": {
            "type": "string",
            "enum": ["duster", "kiger", "unspecified"],
            "description": "Which model the user is asking about, if identifiable."
        }
    },
    required=["model"]
)

EXIT_PRODUCT_EXPERT_TOOL = FunctionSchema(
    name="exit_product_expert_mode",
    description=(
        "CALL THIS when the user's query moves away from vehicle specs/features "
        "back to routing needs (roadside, finance, dealer lookup, radio code) or "
        "when they want to end the call. Restores the base routing system prompt."
    ),
    properties={},
    required=[]
)
```

3. Brochure Data + Prompt Builder

```
DUSTER_BROCHURE = ""
RENAULT_DUSTER - key specs for spoken use:
```

Engines: 1.3 litre Turbo Petrol (156 PS, 254 Nm) with 6-speed manual or X-Tronic CVT;
 1.5 litre diesel (115 PS, 260 Nm) with 6-speed manual.
 Drivetrain: 4x2 and 4x4 (AWD) variants available on the Turbo Petrol.
 Ground clearance: 205 mm. Boot space: 445 litres, expandable with 60:40 split rear seats.
 Safety: up to 6 airbags, ESC, Hill Start Assist, Hill Descent Control, 360 degree camera.
 Interior: 8 inch touchscreen, wireless Android Auto/Apple CarPlay, 7 inch digital cluster,
 wireless phone charger, rain-sensing wipers.
 Design: Y-shaped LED DRLs, muscular wheel arches, roof rails, up to 18 inch alloy wheels.
 """

KIGER_BROCHURE = """
 RENAULT KIGER - key specs for spoken use:
 Engines: 1.0 litre NA petrol (72 PS) manual/AMT; 1.0 litre Turbo petrol (100 PS, 160 Nm)
 manual/CVT.
 Ground clearance: 205 mm. Boot space: 405 litres.
 Safety: up to 6 airbags, ESC, ABS with EBD, rear parking sensors and camera.
 Interior: 8 inch touchscreen, wireless Android Auto/Apple CarPlay, 7 inch digital cluster,
 wireless charging, air purifier, JBL sound system on top trims.
 Design: dual-tone exterior, LED projector headlamps, up to 16 inch diamond-cut alloys.
 """

```
def _build_product_prompt(model: str) -> str:
    if model == "duster":
        brochure = DUSTER_BROCHURE
    elif model == "kiger":
        brochure = KIGER_BROCHURE
    else:
        brochure = DUSTER_BROCHURE + "\n" + KIGER_BROCHURE

    return BASE_SYSTEM_INSTRUCTION + f"""
    You are now operating as the Renault Product Consultant. You have direct
    access to the vehicle brochures.
    DIRECTIVE: When answering features, engine options, or specs, provide rich,
    descriptive, comprehensive explanations using the technical metrics below.
    Speak in fluid, natural paragraphs. Do not read the data as a list - weave
    it into natural spoken sentences.

    [BROCHURE DATA]:
    {brochure}
    """
```

4. Handler Functions & Registration

Critical fix identified: handler signatures must match the project's established convention exactly —
 (function_name, tool_session_id, args, llm, context, result_callback) — not the generic FunctionCallParams style.
 Mismatched signatures raise a TypeError at call time since Pipecat calls every registered handler positionally.

```
async def handle_enter_product_expert(function_name, tool_session_id, args, llm, context, result_callback):
    model = (args or {}).get("model", "unspecified")
    logger.info(f"[ProductExpert] Entering product expert mode for model={model}")
    context.messages[0]["content"] = _build_product_prompt(model)
    await result_callback({"status": "product_expert_mode_active", "model": model})

async def handle_exit_product_expert(function_name, tool_session_id, args, llm, context, result_callback):
    logger.info(f"[ProductExpert] Exiting product expert mode")
    context.messages[0]["content"] = BASE_SYSTEM_INSTRUCTION
    await result_callback({"status": "base_mode_restored"})

llm.register_function("enter_product_expert_mode", handle_enter_product_expert)
llm.register_function("exit_product_expert_mode", handle_exit_product_expert)

tools = ToolsSchema(standard_tools=[
    END_CONVERSATION_TOOL,
    TRANSFER_AGENT_TOOL,
    ENTER_PRODUCT_EXPERT_TOOL,
```

```

    EXIT_PRODUCT_EXPERT_TOOL,
])

initial_messages = [{"role": "system", "content": BASE_SYSTEM_INSTRUCTION}]
context = LLMContext(messages=initial_messages, tools=tools)
context_aggregator = LLMContextAggregatorPair(context=context)

```

5. Final System Prompt (merged language handling)

Combines the original routing-focused BASE_SYSTEM_INSTRUCTION structure with the more explicit, leak-resistant language-state-machine wording. Key change: an explicit 'SILENT LANGUAGE DECISION' rule preventing the model from narrating its language choice out loud (e.g. never say 'since you spoke in English...'), plus a named ENGLISH / HINGLISH state machine with a carry-forward rule for raw data (names, cities, numbers) so isolated tokens don't trigger a false language switch.

```

BASE_SYSTEM_INSTRUCTION = """
You are the Renault Care Assistant, a professional, warm, and highly capable
conversational voice AI. Your core responsibility is to handle inbound customer
calls, greet them, and handle basic routing or simple requests.

STRICT VOICE CHANNEL CONSTRAINTS:
- STRICTLY NO MARKDOWN: Never output asterisks (*), hashtags (#), bullet points,
  bolding, dashes, or text formatting. Output raw, plain, seamless conversational
  text only.
- NO INTERNAL REASONING: Do not output any scratchpads, thoughts, or steps.
  Speak immediately.
- SILENT LANGUAGE DECISION: You must silently determine the correct output
  language for every turn before speaking, using the rules below. Never mention,
  name, describe, or explain this decision in any way. Never say things like
  "since you spoke in English" or "switching to Hinglish" or reference detecting
  a language at all. Just speak directly in the correct language, as if it were
  the only language you know how to speak in that moment.
- PRONUNCIATION: Format phone numbers, VINs, and OTPs digit-by-digit with spaces
  (e.g., "9 8 1 1"). For Hinglish responses, spell out digits phonetically using
  Hindi words (e.g., "9 8 1 1" becomes "nau aath ek ek"). Write acronyms spaced
  out (e.g., "E M I", "V I N").

LANGUAGE STATE MACHINE:
Silently track and apply one of two language states on every turn. This tracking
is internal only and must never surface in your spoken output.

STATE: ENGLISH
- Use when the user's latest turn is primarily in English.
- Speak 100% pure English. Do not use any Hindi or Hinglish tokens
  (no "ji", "achha", "haan", "namaskaar").

STATE: HINGLISH
- Use when the user's latest turn contains Hindi words, Hinglish phrases, or
  Hindi syntax.
- Speak in natural, colloquial Hinglish (Hindi grammar in Latin script, blended
  with common English nouns). Use respectful Hindi pronouns ("aap", "batayein",
  "kijiye").

CARRY-FORWARD RULE:
- If the user's latest turn is only raw data (a name, a city like "Delhi", a
  location like "Wazirpur", a registration plate, or isolated numbers) with no
  other language content, do not switch states. Continue in whichever state you
  used on your previous turn.

GREETING LIFECYCLE:
- If this is the start of the call, say exactly: "Namaskaar! Main Renault ki
  taraf se bol rahi hoon. Kripya batayein main aapki kis prakar sahayata kar
  sakti hoon?"
- Once history exists, never repeat this greeting. Directly address their issue.

```


6. Pipecat LLMContext & System Prompt Reference

LLMContext's constructor only accepts **messages**, **tools**, and **tool_choice** — there is no dedicated `system_prompt` parameter.

```
LLMContext(  
    messages: List[...] | None = None,  
    tools: ToolsSchema | NotGiven = NOT_GIVEN,  
    tool_choice: Literal['none', 'auto', 'required'] | ... = NOT_GIVEN,  
)
```

Two patterns for the system prompt

Pattern A (current approach): put the system prompt as the first entry in messages (e.g. `initial_messages[0]`). Works, but the prompt is just a regular list item — vulnerable to displacement by context summarization, and swapping it means mutating `context.messages[0]` directly.

Pattern B (recommended for Google/Vertex service): set the prompt on the LLM service's `settings.system_instruction` instead, and leave messages empty in the context. This survives context summarization/history replacement, and can be swapped mid-call via an `LLMUpdateSettingsFrame` rather than editing the messages list.

```
llm = GoogleVertexLLMService(  
    project_id=project_id,  
    location="asia-south1",  
    model="gemini-2.5-flash",  
    settings=GoogleVertexLLMService.Settings(  
        temperature=0.1,  
        top_p=0.8,  
        top_k=10,  
        system_instruction=BASE_SYSTEM_INSTRUCTION,    # set base prompt here  
    )  
)  
  
context = LLMContext(tools=tools)    # no system message needed in messages list  
  
# To swap prompts mid-call (e.g. product-expert mode), push a settings-update frame  
# instead of mutating context.messages[0]:  
await task.queue_frames([  
    LLMUpdateSettingsFrame(settings={"system_instruction": _build_product_prompt(model)})  
)
```

Open item to verify

`system_instruction` on `Settings` was confirmed for `GoogleLLMService` in Pipecat's docs. It was **not yet independently confirmed** whether `GoogleVertexLLMService.Settings` exposes the identical field name (Vertex and plain Google AI Studio services sometimes diverge slightly in Pipecat's settings classes). Worth a direct check before refactoring production code around it.

7. Outstanding / Next Steps

- Confirm `GoogleVertexLLMService.Settings` field name for `system_instruction`.
- If confirmed, migrate `BASE_SYSTEM_INSTRUCTION` off `context.messages` and onto the service settings, and rewrite `handle_enter_product_expert` / `handle_exit_product_expert` to use `LLMUpdateSettingsFrame` instead of direct dict mutation.
- Optionally add a nonlocal in `_product_mode` guard to prevent race conditions if enter/exit calls fire in rapid

succession.

- Consider replacing placeholder brochure text with real extracted spec data from the actual Duster/Kiger PDF brochures.